

완 료 보 고 서

UART + FIFO SystemVerilog Verification

UART TX/RX · FIFO · UART+FIFO 통합 SystemVerilog 기능 검증

작 성 : 이찬미

과 정 : 온디바이스 AI 2기

기 간 : 2026.05.12 ~ 2026.05.18

대 상 : UART TX, UART RX, FIFO, UART+FIFO 통합

목 차

1. 개요 (Overview)	3
1.1 검증 목적 및 목표	3
1.2 검증 범위 (Verification Scope)	4
1.3 검증 시나리오 요약	5
1.4 검증 일정표	5
2. 검증 환경 (Verification Environment)	7
2.1 개발 / 설계 환경	7
2.2 검증 환경 구조	7
3. 검증 환경 구조	8
3.1 전체 시스템 구조	8
3.2 UART RX 검증 구조	8
3.3 UART TX 검증 구조	11
3.4 FIFO 검증 구조	13
3.5 UART RX + FIFO RX 검증구조	15
3.6 UART + FIFO LOOPBACK 검증 구조	18
4. 시뮬레이션 결과 (Simulation Results)	21
4.1 UART RX Simulation	21
4.2 UART TX Simulation	24
4.3 FIFO Simulation	27
4.4 UART RX + FIFO RX Simulation	34
4.5 UART + FIFO Loopback Simulation	36
4.6 트러블슈팅 (Trouble Shooting)	40
6. 검증 결과 요약	41
7. 결론 및 고찰	42
참고 문헌	43

1. 개요 (Overview)

1.1 검증 목적 및 목표

본 보고서는 FPGA 기반 UART 통신 시스템에서 사용되는 UART RX, UART TX 및 FIFO 모듈에 대해 SystemVerilog(SV)를 이용한 기능 검증 결과를 기술합니다.

검증 환경은 테스트 데이터 생성, DUT 구동, 출력 데이터 수집 및 결과 비교가 가능하도록 구성하였으며, 랜덤 데이터 기반 테스트를 통해 다양한 입력 조건에서 DUT(Device Under Test)의 동작을 자동으로 검증할 수 있도록 구현하였습니다.

본 프로젝트에서는 UART RX, UART TX, FIFO 모듈에 대한 단독 기능 검증을 수행한 후 UART RX + FIFO RX 통합 검증과 UART RX → FIFO → UART TX 구조의 LOOPBACK 통합 검증을 수행하였습니다. 이를 통해 각 모듈의 개별 동작뿐만 아니라 모듈 간 인터페이스 연동 및 전체 데이터 흐름의 정상 동작 여부를 확인하였습니다.

주요 검증 항목은 다음과 같습니다.

- UART RX 데이터 수신 기능 검증
- UART TX 데이터 송신 기능 검증
- UART Baud Rate Timing 검증
- FIFO Push/Pop 기능 검증
- FIFO Full/Empty 상태 검증
- UART RX + FIFO RX 데이터 연동 검증
- UART RX → FIFO → UART TX LOOPBACK 검증
- Random Data 기반 데이터 무결성(Data Integrity) 검증

본 검증의 목적은 RTL 설계 코드가 UART 프로토콜 규격(9600bps, 8-N-1 Format)과 FIFO 동작 규격(Push/Pop, Full/Empty Flag)을 올바르게 구현하였는지 확인하고, 통합 시스템 환경에서 데이터 손실 없이 안정적으로 동작함을 검증하는 데 있습니다.

1.2 검증 범위 (Verification Scope)

검증 시나리오	대상 모듈	검증 내용
UART RX 검증	UART_RX	Random 데이터를 UART RX로 입력하여 rx_done 신호 발생 여부 및 rx_data 출력 데이터의 정합성을 확인합니다.
UART TX 검증	FIFO_RX / FIFO_TX	Random 데이터를 입력하여 직렬 송신 동작을 확인하고 송신 완료 후 데이터 복원 결과를 검증합니다.
FIFO Full 상태 검증	FIFO	FIFO DEPTH까지 연속 Push를 수행하여 Full Flag가 정상적으로 발생하는지 확인합니다.
FIFO 데이터 무결성 검증	FIFO	Random Push/Pop 시나리오를 적용하여 데이터 순서 보존 여부와 데이터 유실 여부를 검증합니다.
UART RX + FIFO RX 통합 검증	UART_RX + FIFO_RX	UART RX에서 수신한 데이터가 FIFO에 정상 저장되는지 확인하고 Pop 데이터와 RX 데이터를 비교하여 정합성을 검증합니다.
UART + FIFO LOOPBACK 검증	UART + FIFO	UART RX → FIFO → UART TX 전체 데이터 흐름을 검증하고 입력 데이터와 최종 출력 데이터의 일치 여부를 확인합니다.

1.3 검증 시나리오 요약

본 프로젝트는 UART RX, UART TX, FIFO 모듈의 단독 기능 검증을 수행한 후 UART RX + FIFO RX 통합 검증 및 UART + FIFO LOOPBACK 통합 검증을 단계적으로 수행하였습니다.

검증은 Random Data 기반으로 진행하였으며 데이터 정합성, Baud Rate Timing, FIFO 데이터 순서 보존 및 전체 데이터 무결성을 중점적으로 확인하였습니다.

검증 수행 순서는 다음과 같습니다.

1. UART RX 기능 검증
2. UART TX 기능 검증
3. FIFO 기능 검증
4. UART RX + FIFO RX 통합 검증
5. UART + FIFO LOOPBACK 통합 검증

각 단계별 검증이 완료된 후 다음 단계로 진행하여 모듈 단위 기능과 시스템 통합 동작을 순차적으로 검증하였습니다.

1.4 검증 일정표

단계	기간	주요 작업
프로젝트 선정 및 기획 단계	26.05.12 ~ 26.05.13	요구사항 정리, 검증 기획, 검증 환경 구성
개발 및 구현 단계	26.05.14 ~ 26.05.15	UART RX 구현, UART TX 구현, FIFO 구현, UART RX+FIFO RX 구현, UART + FIFO LOOPBACK 구현

단계	기간	주요 작업
검증 및 통합 단계	26.05.15 ~ 26.05.16	UART RX 검증, UART TX 검증, FIFO 검증, UART RX + FIFO RX 검증, UART + FIFO LOOPBACK 검증, 디버깅
발표 및 문서화 단계	26.05.17 ~ 26.05.18	시뮬레이션 결과 정리, 문서 작성, 발표 준비, 최종 점검, 발표

2. 검증 환경 (Verification Environment)

2.1 개발 / 설계 환경

분류	내용
설계 언어	Verilog HDL
검증 언어	SystemVerilog (SV)
개발 도구	Xilinx Vivado 2020.2
시뮬레이터	Vivado Simulator

2.2 검증 환경 구조

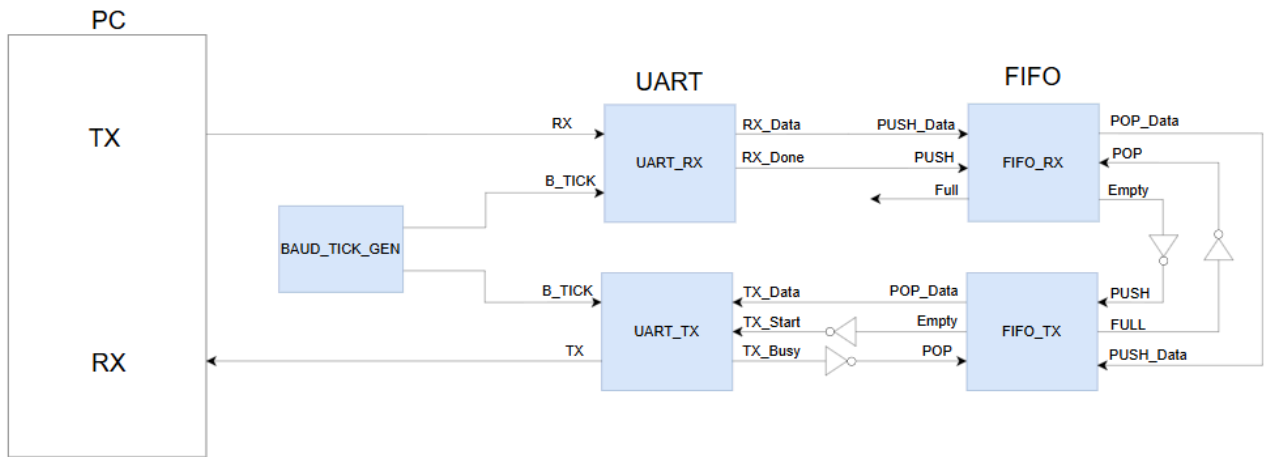
본 검증에서는 SystemVerilog의 클래스 기반 객체지향 기능을 활용하여 Generator-Driver-Monitor-Scoreboard 구조의 자동화 검증 환경을 구성하였습니다. 각 컴포넌트의 역할은 다음과 같습니다.

컴포넌트	역할 및 동작
Generator	검증에 사용할 Transaction 객체를 생성하고 테스트 데이터를 생성합니다. 생성된 데이터는 Driver와 Scoreboard로 전달됩니다.
Driver	Generator로부터 전달받은 Transaction 데이터를 DUT 입력 신호 형태로 변환하여 DUT에 인가합니다. UART 검증 시 Start Bit, Data Bit, Stop Bit를 포함한 직렬 데이터를 생성합니다.
Monitor	DUT 출력 신호를 관찰하여 수신된 데이터를 추출하고 검증 결과 비교를 위해 Scoreboard로 전달합니다.
Scoreboard	기대값(Expected Data)과 실제 출력값(Actual Data)을 비교하여 데이터 정합성을 검증하고 Pass/Fail 결과를 출력합니다.

3. 검증 환경 구조

3.1 전체 시스템 구조

UART RX에서 수신된 데이터는 FIFO에 저장되며, FIFO에 저장된 데이터는 UART TX를 통해 다시 송신됩니다. 본 시스템은 UART RX, FIFO, UART TX 모듈로 구성되며, UART 통신을 통해 수신된 데이터가 FIFO를 거쳐 다시 송신되는 LOOPBACK 구조로 동작합니다.



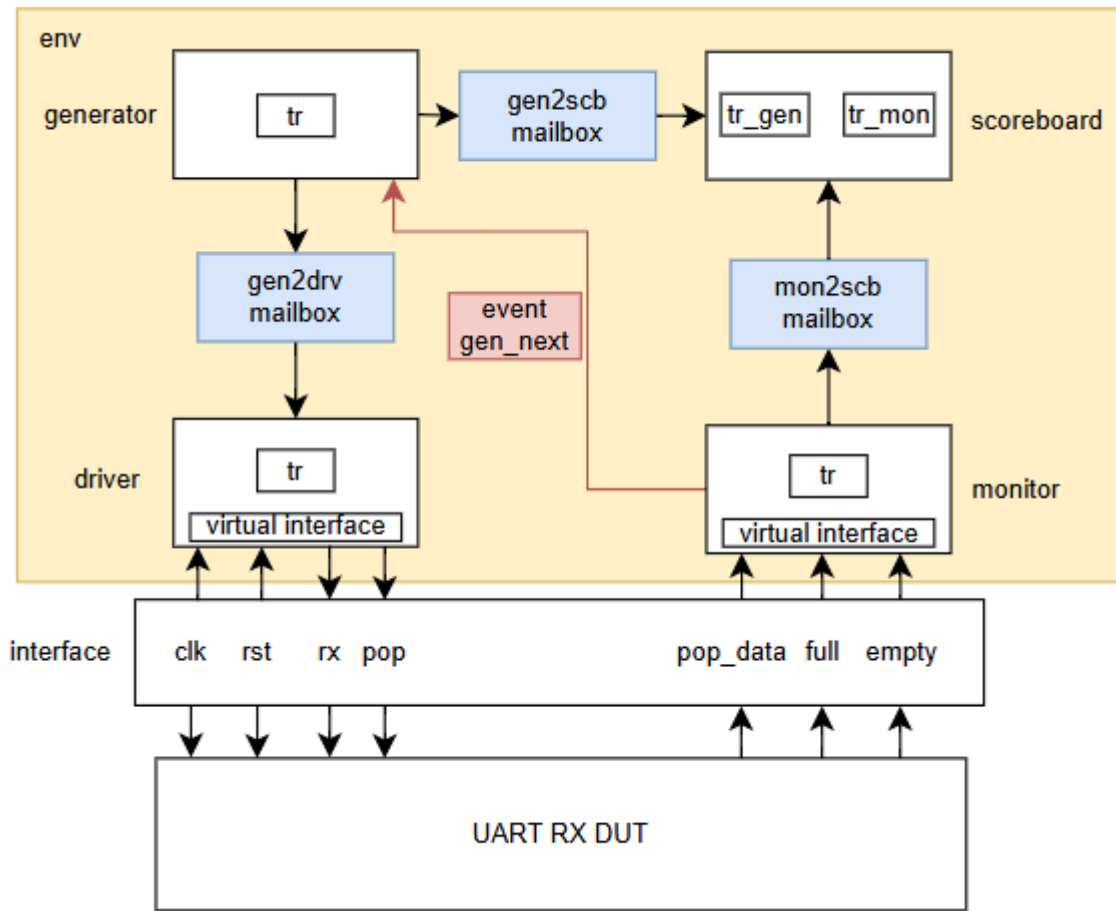
[그림 3-1] 전체 System Block diagram

3.2 UART RX 검증 구조

(1) UART RX 검증 대상

UART RX는 UART 프로토콜 형식으로 입력되는 직렬 데이터를 수신하여 병렬 데이터(rx_data)와 수신 완료 신호(rx_done)를 출력하는 모듈입니다. 본 검증에서는 UART RX의 데이터 수신 기능과 수신 완료 신호 발생 여부를 확인하였습니다.

(2) UART RX Verification Architecture



[그림 3-2] UART RX Systemverilog testbench

그림 3-2는 UART RX 검증을 위한 SystemVerilog Testbench 구조입니다. Generator는 Random 데이터를 생성하여 Driver와 Scoreboard에 전달합니다. Driver는 생성된 데이터를 UART 프로토콜 형식의 직렬 데이터로 변환하여 DUT에 입력합니다. Monitor는 DUT의 출력 신호를 수집하여 Scoreboard로 전달하며, Scoreboard는 Generator가 생성한 기대값과 Monitor가 수집한 실제 출력값을 비교하여 데이터 정합성을 검증합니다.

본 검증 환경은 UART RX가 입력된 직렬 데이터를 정상적으로 병렬 데이터로 복원하고, 수신 완료 신호(rx_done)를 올바르게 발생시키는지 확인하도록 구성하였습니다.

(3) UART RX Drive 타이밍

Driver는 UART 송신기 역할을 수행하며, 내부 시스템 클럭과 독립적으로 Baud Rate 타이밍에 맞추어 직렬 데이터를 DUT에 입력합니다. 검증 데이터는 UART 프레임 형식으로 변환되어 Start Bit, Data Bit, Stop Bit 순서로 전송됩니다.

항목	내용
프레임 구성	uart_frame = {1'b1(stop), rx_data[7:0], 1'b0(start)}
Baud Rate	#104160 (9600bps 기준)
drive 방식	시스템 클럭과 독립적인 비동기 직렬 데이터 전송
drive 순서	Start bit(0) → Data bit[0]~[7] → Stop bit(1)

(4) UART RX Monitor 타이밍

Monitor는 UART RX의 수신 완료 신호인 rx_done을 감시하며, 데이터 수신 완료 후 안정화된 시점에서 rx_data를 수집합니다. 수집된 데이터는 Scoreboard로 전달되며, 검증 완료 후 Event를 발생시켜 다음 Transaction 검증이 진행되도록 구성하였습니다.

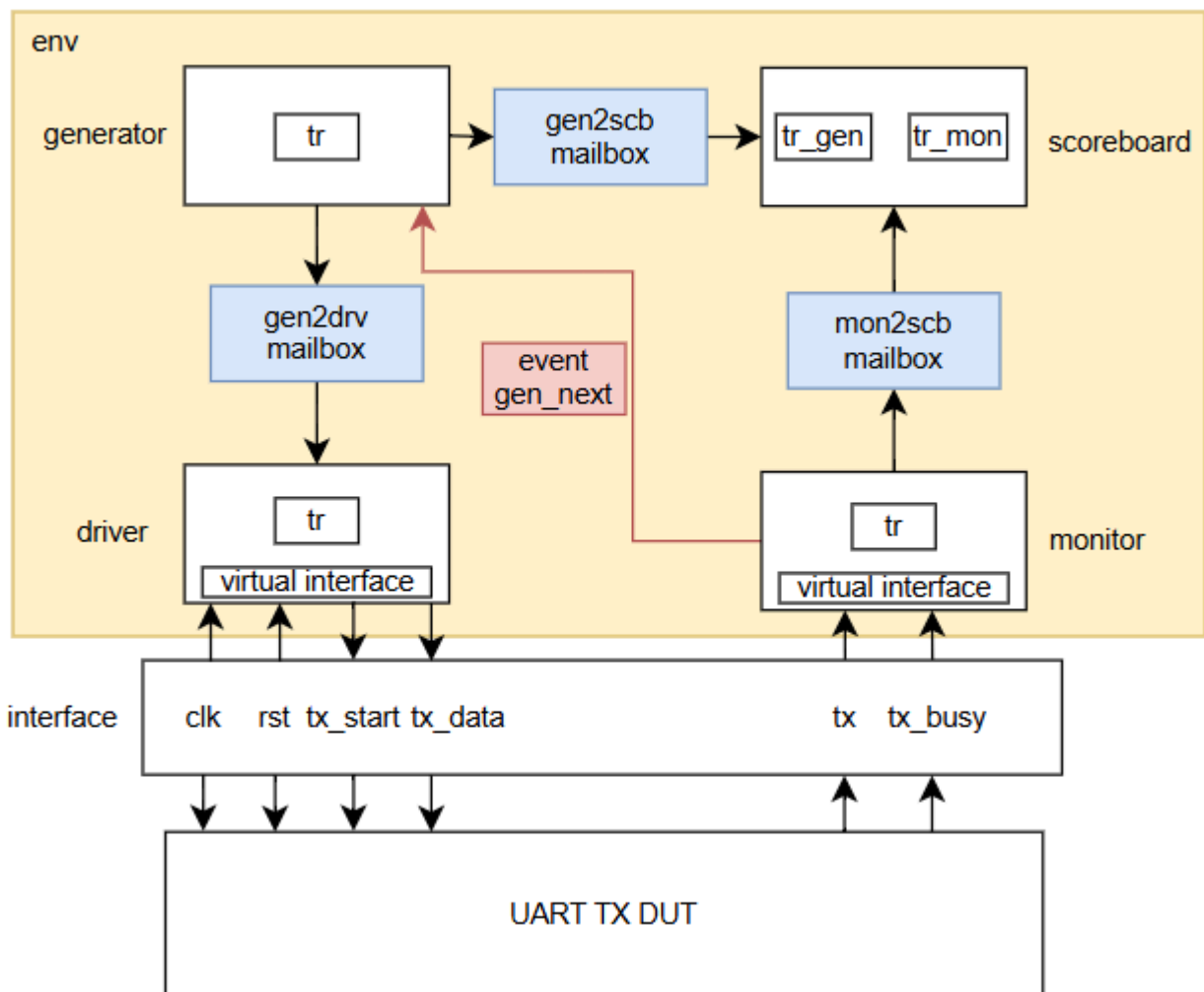
항목	내용
감지 신호	@(posedge rx_done)
데이터 포착 시점	@(negedge clk)
수집 데이터	rx_data
데이터 전달	mon2scb_mbox.put(tr)
동기화 방식	->event_gen_next 발생
검증 목적	수신 완료 데이터와 기대값 간 정합성 검증

3.3 UART TX 검증 구조

(1) UART TX 검증 대상

UART TX는 입력된 병렬 데이터를 UART 프로토콜 형식의 직렬 데이터로 변환하여 송신하는 모듈입니다. 본 검증에서는 입력 데이터가 UART 프레임 형식으로 정상 송신되는지 확인하고, Baud Rate 타이밍에 따른 직렬 데이터 출력 및 송신 완료 동작을 검증하였습니다.

(2) UART TX Verification Architecture



[그림 3-3] UART TX Systemverilog testbench

그림 3-3은 UART TX 검증을 위한 SystemVerilog Testbench 구조입니다.

Generator는 Random 데이터를 생성하여 Driver와 Scoreboard에 전달합니다.

Driver는 **DUT**에 송신 데이터와 송신 시작 신호를 입력하며, **Monitor**는 **UART TX**의 직렬 출력 신호를 수집하여 원본 데이터를 복원합니다.

Scoreboard는 **Generator**가 생성한 기대값과 **Monitor**가 복원한 실제 데이터를 비교하여 데이터 정합성을 검증합니다. 또한 **Monitor**와 **Generator** 간 **Event** 동기화를 통해 이전 프레임 검증 완료 후 다음 **Transaction**이 생성되도록 구성하였습니다.

(3) UART TX Drive 타이밍

Driver는 **DUT**의 병렬 데이터 입력 포트에 데이터를 인가하고 **tx_start** 신호를 발생시켜 **UART** 송신을 시작합니다. 또한 이전 프레임 전송이 완료된 후 다음 데이터가 전송되도록 **tx_busy** 상태를 확인하여 송신 순서를 제어합니다.

항목	내용
입력 데이터	tx_data[7:0]
송신 시작	tx_start = 1
전송 조건	wait(!tx_busy)
Drive 방식	병렬 데이터 입력 후 송신 시작 신호 발생
검증 목적	송신 순서 보장 및 데이터 유실 방지

(4) UART TX Monitor 타이밍

Monitor는 **UART TX** 출력 신호인 **tx**를 감시하여 **Start Bit**를 검출한 후 **UART** 프레임을 복원합니다. 복원된 데이터는 **Scoreboard**로 전달되며, 검증 완료 후 **Event**를 발생시켜 다음 **Transaction** 검증이 수행되도록 구성하였습니다.

항목	내용
감지 신호	@(negedge tx)
검출 대상	Start Bit
데이터 복원	Data Bit[0]~[7] 샘플링
데이터 전달	mon2scb_mbox.put(tr)

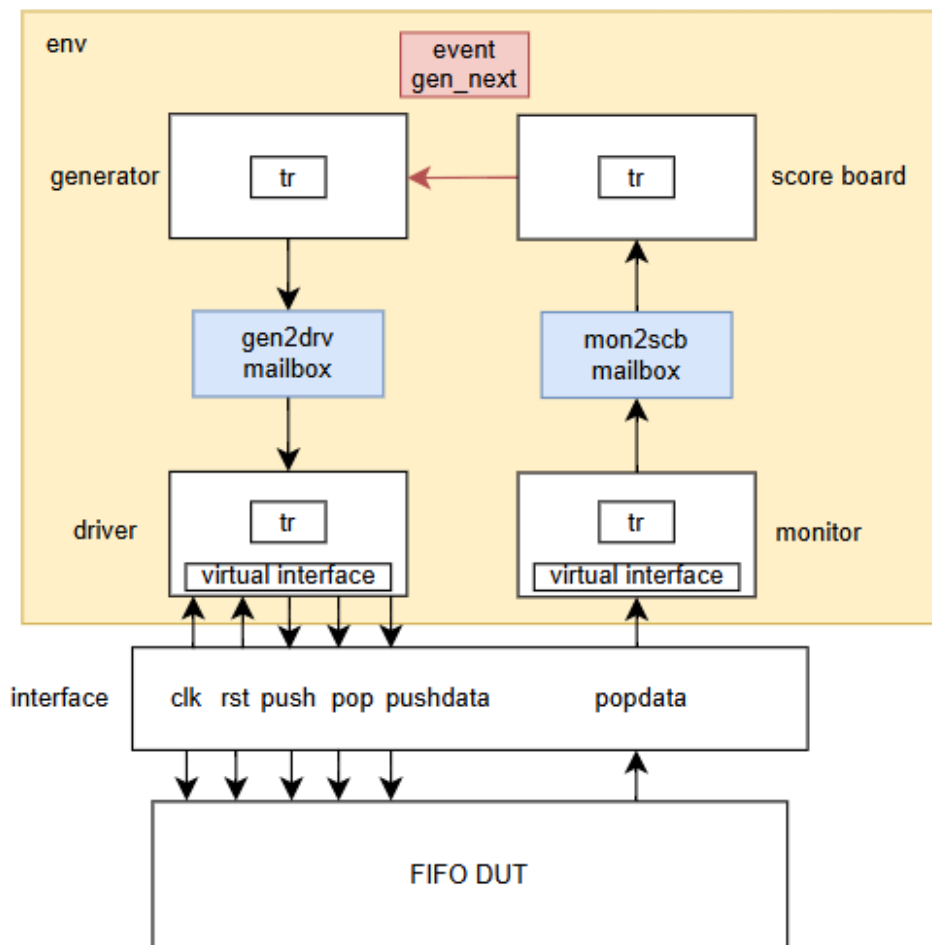
동기화 방식	->event_gen_next 발생
검증 목적	송신 데이터와 복원 데이터 간 정합성 검증

3.4 FIFO 검증 구조

(1) FIFO TX 검증 대상

FIFO는 입력된 데이터를 임시 저장하고 순차적으로 출력하는 First In First Out 구조의 버퍼 모듈입니다. 본 검증에서는 Push/Pop 동작, Full 상태 검출 기능 및 Random Push/Pop 시 데이터 정합성과 순서 보존 여부를 검증하였습니다.

(2) FIFO Verification Architecture



[그림 3-4] FIFO Systemverilog testbench

그림 3-4는 FIFO 검증을 위한 SystemVerilog Testbench 구조입니다.

Generator는 Random 데이터를 생성하여 Driver로 전달하고, Driver는 FIFO의 Push 및 Pop 동작을 제어하여 DUT에 데이터를 입력합니다.

Monitor는 FIFO 출력 데이터와 상태 신호를 수집하여 Scoreboard로 전달합니다. Scoreboard는 내부 Queue에 저장된 기대 데이터와 FIFO 출력 데이터를 비교하여 데이터 정합성과 데이터 순서 보존 여부를 검증합니다.

본 검증 환경은 FIFO의 데이터 저장 및 출력 기능, Full 상태 검출 기능, 그리고 FIFO 특성인 선입선출(FIFO) 동작이 정상적으로 수행되는지 확인하도록 구성하였습니다.

(3) FIFO Drive 타이밍

Driver는 FIFO 입력 데이터(push_data)와 제어 신호(push, pop)를 생성하여 DUT에 입력합니다. 데이터 및 제어 신호는 클럭 상승 에지 이후 1ns 지연된 시점에 인가하여 DUT가 Race condition 없이 안정적으로 신호를 수신할 수 있도록 구성하였습니다.

항목	내용
입력 데이터	push_data[7:0]
제어 신호	push, pop
Drive 시점	@(posedge clk) 이후 #1
Drive 방식	데이터 및 제어 신호 동시 인가
검증 목적	Push/Pop 동작 및 데이터 저장 기능 검증

(4) FIFO Monitor 타이밍

Monitor는 FIFO 출력 데이터(pop_data)를 클럭 하강 에지 시점에 수집하여 Scoreboard로 전달합니다. Scoreboard는 내부 Queue에 저장된 기대 데이터와 실제 출력 데이터를 비교하여 데이터 정합성과 FIFO 순서 보존 여부를 검증합니다.

항목	내용
감지 신호	@(negedge clk)
수집 데이터	pop_data
데이터 전달	mon2scb_mbox.put(tr)
비교 방식	Queue 기반 Expected Data 비교
검증 목적	데이터 정합성 및 FIFO 순서 보존 검증

3.5 UART RX + FIFO RX 검증구조

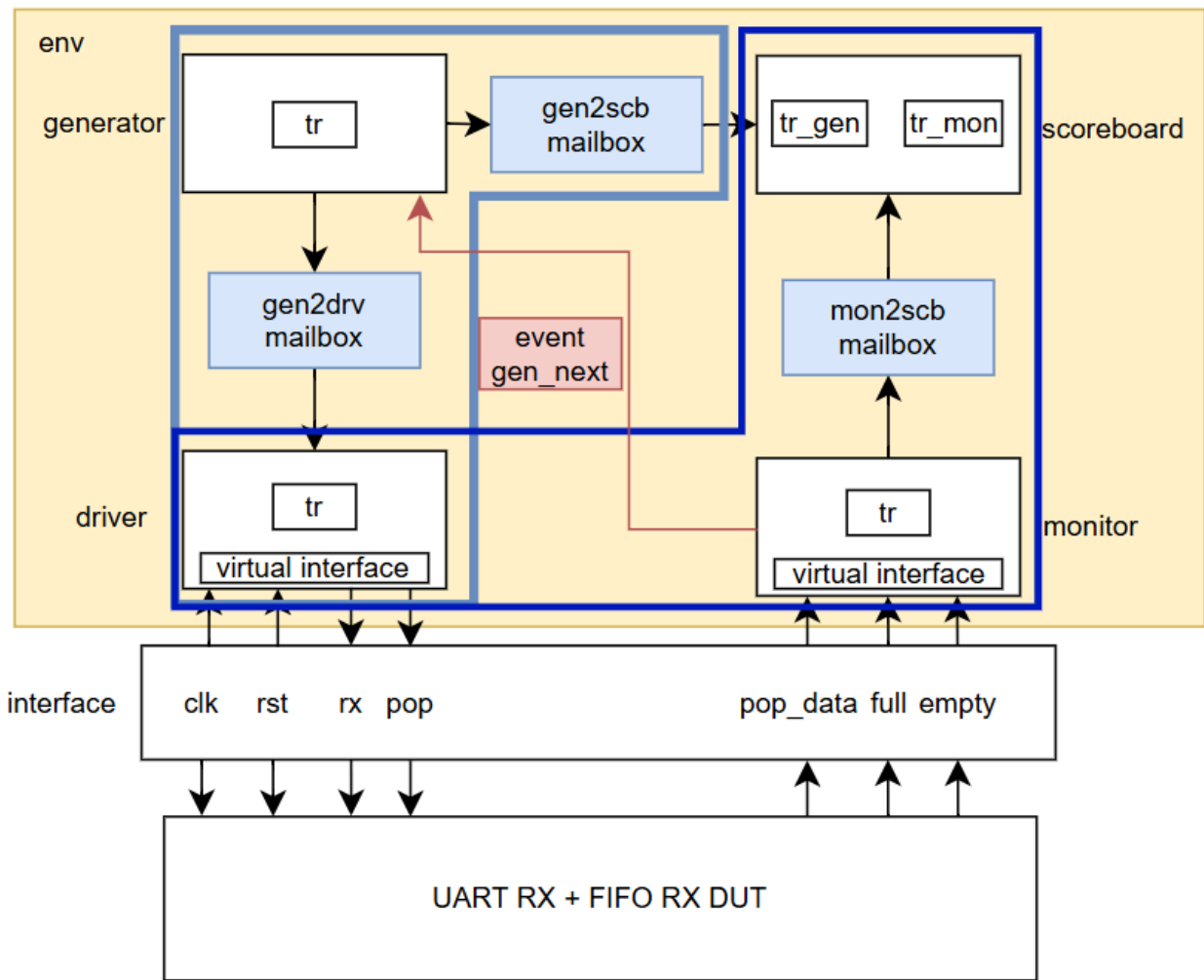
(1) UART RX + FIFO RX 검증 대상

본 검증 구조는 UART RX를 통해 수신된 직렬 데이터를 병렬 데이터로 복원한 뒤 FIFO에 저장하고, FIFO 출력까지의 전체 데이터 흐름의 정합성을 검증하는 구조입니다.

UART RX 단계에서는 Start Bit 검출 및 Bit Sampling을 통해 8-bit 데이터를 복원하며, 복원된 데이터는 FIFO 입력으로 전달됩니다. 이후 FIFO는 해당 데이터를 저장 및 출력하면서 선입선출(FIFO) 동작을 수행합니다.

본 검증의 핵심은 UART RX 데이터 복원 정확성 + FIFO 저장 및 출력 순서 보장입니다.

(2) UART RX + FIFO RX Verification Architecture



[그림 3-5] UART RX + FIFO RX Systemverilog testbench

그림 3-5는 UART RX 입력을 통해 수신된 직렬 데이터가 FIFO로 전달되고, FIFO 출력까지 이어지는 전체 수신 경로를 검증하기 위한 SystemVerilog Testbench 구조입니다.

Generator는 테스트 데이터를 생성하여 Driver로 전달하며, 동시에 기대 데이터(expected data)를 Scoreboard로 mailbox를 통해 전달합니다.

Driver는 UART RX 입력 신호(rx)를 DUT에 인가하여 UART 수신 동작을 발생시키고, 복원된 데이터가 FIFO로 저장된 뒤 정상적으로 출력되도록 전체 데이터 흐름을 유도합니다. 또한 FIFO의 출력 동작은 pop 제어 신호에 의해 수행됩니다.

UART RX는 입력된 직렬 데이터(rx)를 **Start Bit** 기준으로 샘플링하여 **8-bit** 병렬 데이터로 복원하고, 해당 데이터는 **FIFO** 입력으로 전달되어 저장됩니다. 이후 **FIFO**는 **pop** 동작에 의해 데이터를 순차적으로 출력합니다. **Monitor**는 **FIFO** 출력 데이터(pop_data)를 수집하여 **Scoreboard**로 mailbox를 통해 전달하며, **Scoreboard**는 **Generator**가 전달한 기대 데이터와 **Monitor**가 전달한 실제 **FIFO** 출력 데이터를 비교하여 전체 수신 경로의 데이터 정합성을 검증합니다.

본 구조에서는 **UART RX → FIFO RX → FIFO POP**까지 이어지는 단일 데이터 스트림 기반 검증을 수행하며, **Driver**는 **RX** 입력 구동과 **FIFO POP** 동작 제어를 함께 수행하여 데이터 흐름을 유지합니다.

(3) UART RX + FIFO RX Drive 타이밍

본 검증은 **UART RX** 입력을 통한 데이터 수신과 **FIFO** 출력 검증을 하나의 데이터 흐름으로 구성하며, **RX** 입력 시나리오와 **FIFO POP** 시나리오로 분리하여 동작을 정의하였습니다.

RX 입력 시나리오는 **Generator–Driver** 구간에서 수행되며, **Generator**가 생성한 **UART** 데이터가 **baud** 기준으로 **rx** 신호로 변환되어 **DUT**에 인가된다. 이를 통해 **UART RX** 수신 및 **FIFO write** 경로가 형성됩니다.

FIFO POP 시나리오는 **Driver–Monitor–Scoreboard** 구간에서 수행되며, **Driver**는 **FIFO**의 출력 동작을 위해 **pop** 신호를 생성하고, **Monitor**는 해당 시점에서 **FIFO** 출력 데이터(pop_data)를 클럭 기준으로 캡처하여 **Scoreboard**로 전달합니다. **Scoreboard**는 **Generator**의 기대 데이터와 **Monitor**가 전달한 실제 **FIFO** 출력 데이터를 비교하여 데이터 정합성을 검증합니다.

전체 데이터 흐름은 **UART RX → FIFO write → FIFO pop → Scoreboard 비교** 구조로 구성되며, 입력과 출력 검증 경로를 분리하여 **FIFO**의 **write/read** 동작을 독립적으로 검증할 수 있도록 설계하였습니다.

<시나리오 구성>

항목	내용
RX 입력 시나리오	Generator-Driver 구간에서 rx baud 기준 drive → FIFO write
POP 검증 시나리오	Driver-Monitor-Scoreboard 구간에서 pop 제어 → FIFO read
Monitor 동작	pop 유효 시점에서 pop_data 캡처 후 Scoreboard 전달
Scoreboard	expected vs actual FIFO 출력 비교

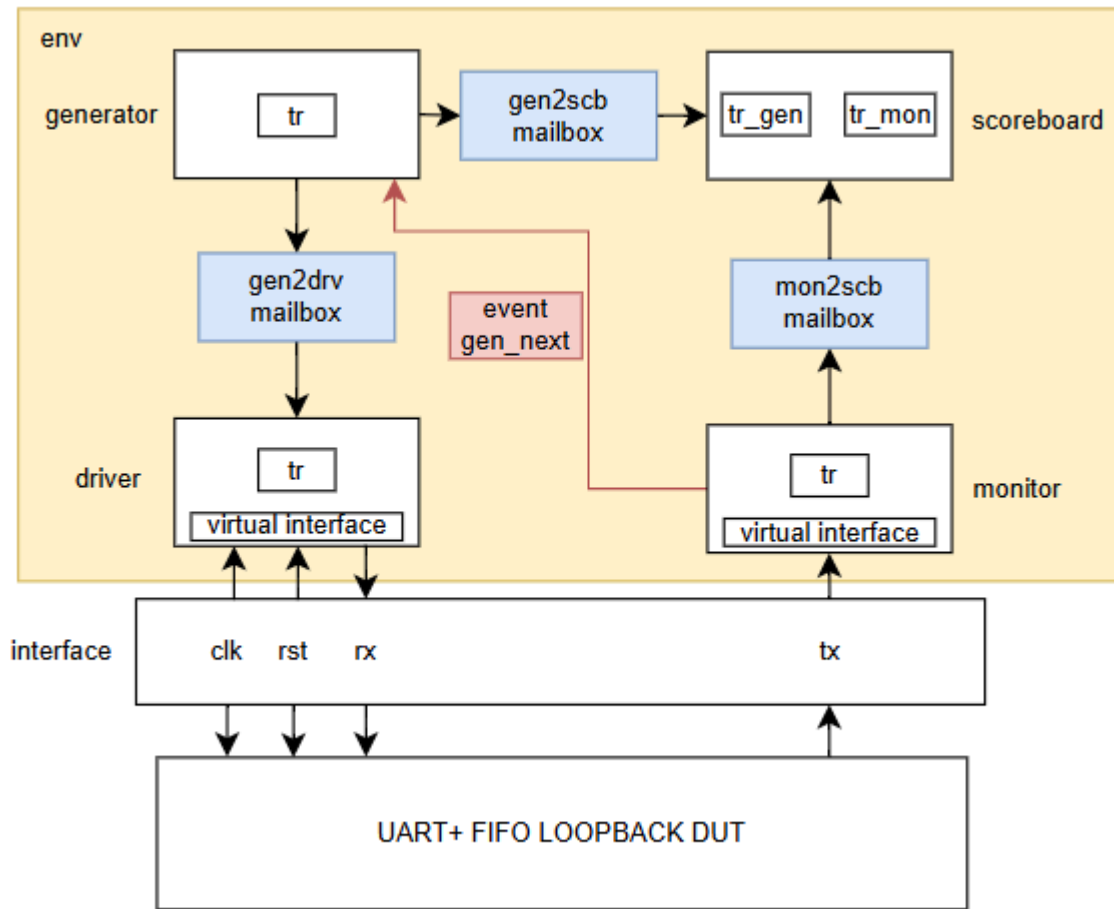
3.6 UART + FIFO LOOPBACK 검증 구조

(1) UART RX + FIFO Loopback 검증 대상

본 검증 구조는 UART RX로 수신된 직렬 데이터가 FIFO에 저장된 후 FIFO TX를 통해 다시 UART TX로 출력되는 전체 loopback 경로를 대상으로 합니다.

즉, UART RX → FIFO RX(write) → FIFO TX(read) → UART TX 출력까지의 end-to-end 데이터 흐름에서 데이터 정합성과 FIFO 순서 보존 여부를 검증합니다.

(2) UART + FIFO Loopback Verification Architecture



[그림 3-6] UART + FIFO LOOPBACK Systemverilog testbench

그림 3-6은 UART + FIFO Loopback 검증을 위한 SystemVerilog Testbench 구조입니다.

Generator는 UART RX 입력 기준의 테스트 데이터를 생성하여 Driver로 전달하며, 동시에 기대 데이터(expected data)를 Scoreboard로 mailbox를 통해 전달합니다.

UART RX는 입력된 직렬 데이터(rx)를 Start Bit 기준으로 샘플링하여 8-bit 병렬 데이터로 복원하고, 해당 데이터는 FIFO RX(write) 경로로 전달됩니다. FIFO TX(read) 동작을 통해 데이터가 출력되며, 최종적으로 UART TX를 통해 직렬 데이터로 변환되어 외부로 출력됩니다.

Monitor는 **UART TX** 출력 신호(tx)의 하강 엣지를 감시하여 복원된 데이터를 생성하고 **Scoreboard**로 **mailbox**를 통해 전달하며, **Scoreboard**는 **Generator**에서 전달된 기대 데이터와 **Monitor**에서 전달된 실제 **UART TX** 출력 데이터를 직접 비교하여 **end-to-end loopback** 경로의 데이터 정합성을 검증합니다.

(3) Scoreboard 동작 방식

Scoreboard는 **Generator**와 **Monitor**로부터 **mailbox**를 통해 각각 데이터를 수신하여 직접 비교하는 구조로 동작합니다.

Generator는 **expected data**를 **Scoreboard**로 전달하고, **Monitor**는 **actual UART TX output** 데이터를 **Scoreboard**로 전달합니다.

본 **loopback** 구조에서는 별도의 **Queue** 기반 정렬 없이 **Generator-Monitor** 간 **1:1 mailbox** 기반 비교를 통해 **transaction** 단위 정합성을 검증합니다. 이를 통해 **UART RX → FIFO → UART TX**까지의 전체 경로에서 데이터 손실 및 순서 오류 여부를 확인합니다.

(4) 검증 기준

항목	내용
비교 대상	expected data vs UART TX output data
비교 방식	mailbox 기반 1:1 direct comparison
검증 범위	UART RX → FIFO RX → FIFO TX → UART TX
검증 목적	end-to-end loopback 데이터 정합성 검증

4. 시뮬레이션 결과 (Simulation Results)

4.1 UART RX Simulation

UART RX 검증은 Preset과 Randomize 두 가지 시나리오로 구성됩니다.

항목	시나리오
Preset	Reset 이후 idle 상태 및 초기값 정상 동작 확인
Randomize	랜덤 데이터 500개 전송을 통한 데이터 정합성 검증

(1) Preset 시나리오 결과

Reset 이후 UART RX 초기 상태를 확인하였으며 rx idle 상태와 내부 signal 초기화가 정상적으로 수행됨을 검증하였습니다.



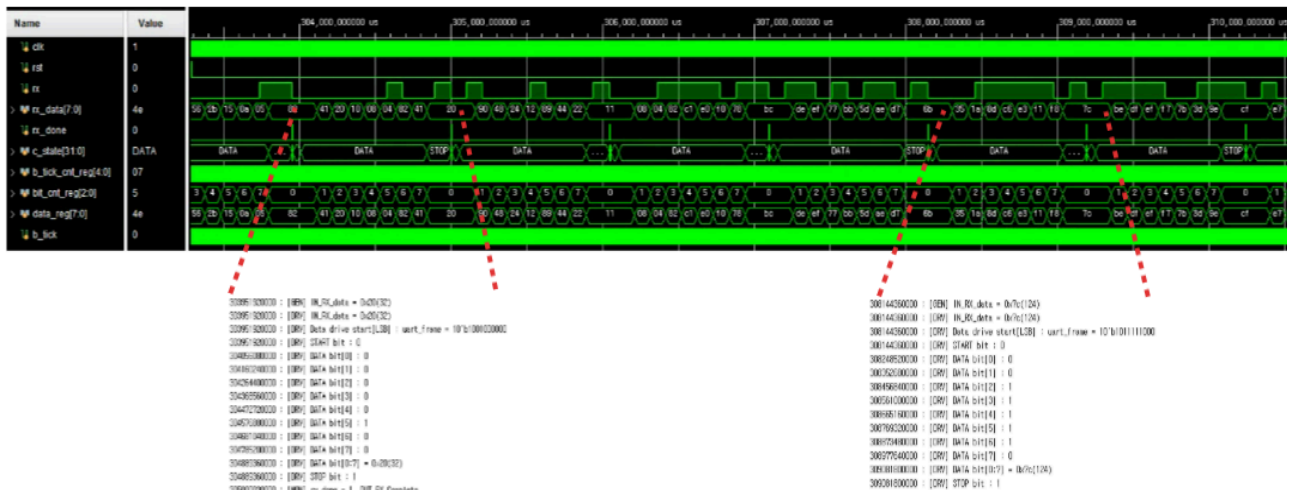
```
[DRV Assert] reset pass : rx = 1
[DRV Assert] reset pass : rx_data = 0x00
[DRV Assert] reset pass : rx_done = 0
```

[그림 4-1] UART RX 시나리오 1(Preset)

(2) Random 시나리오 결과

Generator를 통해 생성된 랜덤 데이터 500개를 UART RX로 입력하였으며, 수신된 데이터는 Scoreboard를 통해 기대값과 비교되었습니다.

데이터는 transaction 단위로 비교되며 UART RX 수신 정확성과 데이터 정합성을 검증합니다.



```

1048130000 : [GEN] IN_RX_data = 0xee(238)
1048130000 : [DRV] IN_RX_data = 0xee(238)
1048130000 : [DRV] Data drive start[LSB] : uart_frame = 10'b1111011100
1048130000 : [DRV] START bit : 0
1152290000 : [DRV] DATA bit[0] : 0
1256450000 : [DRV] DATA bit[1] : 1
1360610000 : [DRV] DATA bit[2] : 1
1464770000 : [DRV] DATA bit[3] : 1
1568930000 : [DRV] DATA bit[4] : 0
1673090000 : [DRV] DATA bit[5] : 1
1777250000 : [DRV] DATA bit[6] : 1
1881410000 : [DRV] DATA bit[7] : 1
1985570000 : [DRV] DATA bit[0:7] = 0xee(238)
1985570000 : [DRV] STOP bit : 1
2096240000 : [MON] rx_done = 1, DUT RX Complete
2096240000 : [MON] DUT_RX_data = 0xee(238)
2096240000 : [SCB] PASS !! Expected = 0xee(238), GOT Rx_data = 0xee(238)
    
```

[그림 4-2] UART RX 시나리오2(Random)

(3) Scoreboard 결과

UART RX 검증은 Generator 기반 expected data와 DUT output 데이터를 Scoreboard에서 비교하여 수행됩니다.

```
-----
** UART RX Verification **
** total test num = 500 **
** pass test num  = 500 **
** fail test num  =  0 **
*****
```

[그림 4-3] UART RX Scoreboard 결과 (PASS)

Scoreboard 기준 모든 transaction이 일치하여 UART RX 기능이 정상적으로 동작함을 확인하였습니다.

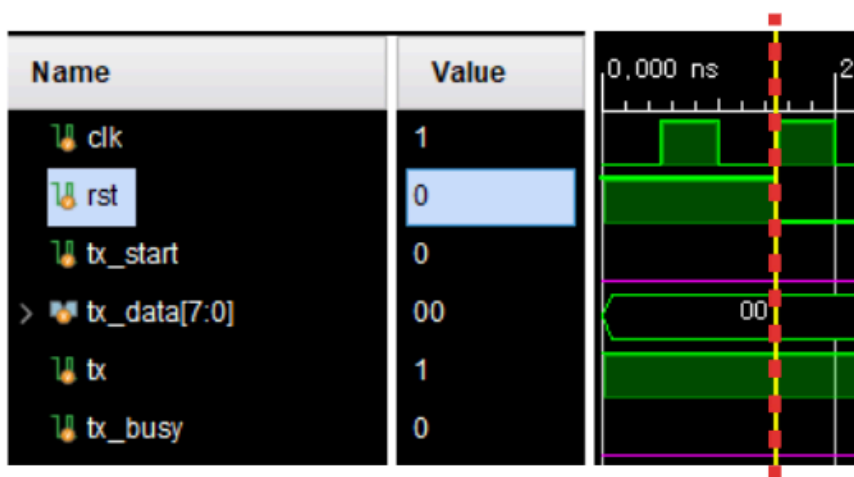
4.2 UART TX Simulation

UART TX 검증은 Preset과 Randomize 두 가지 시나리오로 구성됩니다.

항목	시나리오
Preset	tx_start, tx_busy 초기 상태 및 idle 상태 동작 확인
Randomize	랜덤 데이터 500개 송신을 통한 UART TX 데이터 정합성 검증

(1) Preset 시나리오 결과

Reset 이후 UART TX 초기 상태를 확인하였으며 tx idle 상태와 내부 signal 초기화가 정상적으로 수행됨을 검증하였습니다.



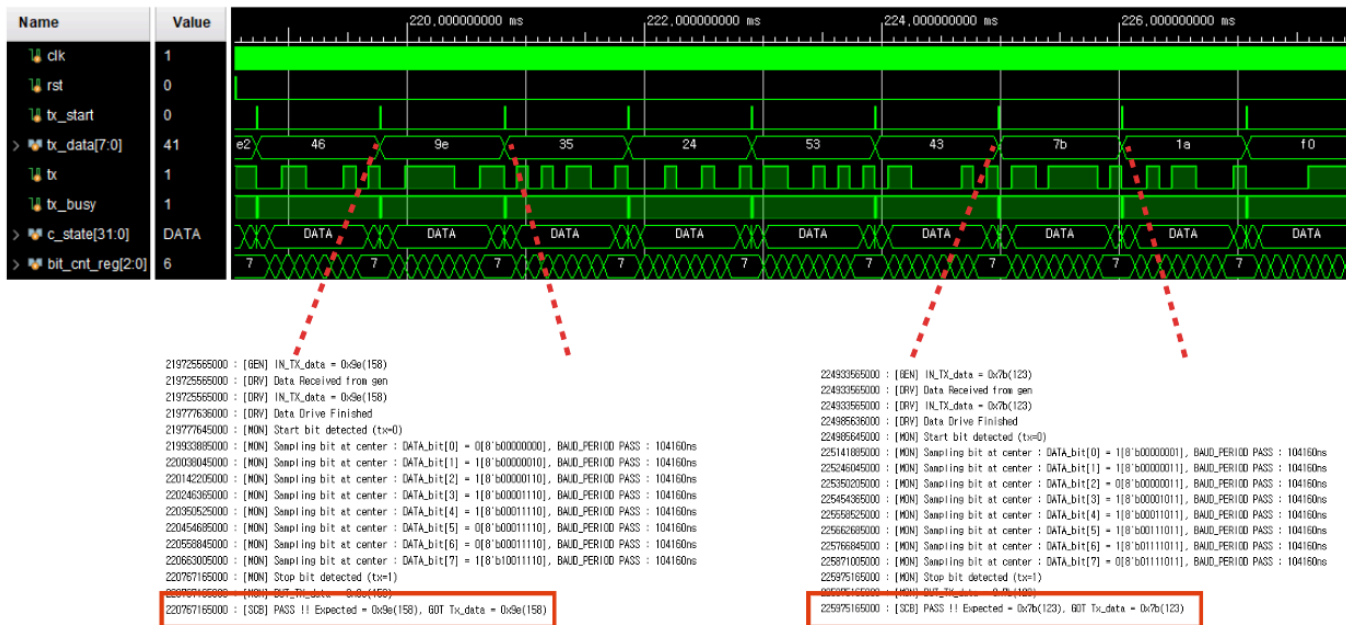
```
[DRV Assert] reset pass : tx = 1
[DRV Assert] reset pass : tx_busy = 0
```

[그림 4-4] UART TX 시나리오1(Preset)

(2) Random 시나리오 결과

Generator에서 생성된 랜덤 데이터 500개를 UART TX 입력(tx_data)으로 전달하여 송신 동작을 수행하였으며, 송신된 직렬 데이터는 Monitor에서 복원하여 Scoreboard를 통해 기대값과 비교되었습니다.

UART TX는 병렬 데이터를 직렬 데이터로 변환하여 전송하며, 각 transaction은 tx_busy 상태를 기준으로 순차적으로 처리됩니다. 이를 통해 데이터 손실 없이 정확한 송신이 수행되는지를 검증합니다.



```

989565000 : [GEN] IN_TX_data = 0xee(238)
989565000 : [DRV] Data Received from gen
989565000 : [DRV] IN_TX_data = 0xee(238)
1041636000 : [DRV] Data Drive Finished
1041645000 : [MON] Start bit detected (tx=0)
1197885000 : [MON] Sampling bit at center : DATA_bit[0] = 0[8'b00000000], BIT Period PASS : 104160ns
1302045000 : [MON] Sampling bit at center : DATA_bit[1] = 1[8'b00000010], BIT Period PASS : 104160ns
1406205000 : [MON] Sampling bit at center : DATA_bit[2] = 1[8'b00000110], BIT Period PASS : 104160ns
1510365000 : [MON] Sampling bit at center : DATA_bit[3] = 1[8'b00001110], BIT Period PASS : 104160ns
1614525000 : [MON] Sampling bit at center : DATA_bit[4] = 0[8'b00001110], BIT Period PASS : 104160ns
1718685000 : [MON] Sampling bit at center : DATA_bit[5] = 1[8'b00101110], BIT Period PASS : 104160ns
1822845000 : [MON] Sampling bit at center : DATA_bit[6] = 1[8'b01101110], BIT Period PASS : 104160ns
1927005000 : [MON] Sampling bit at center : DATA_bit[7] = 1[8'b11101110], BIT Period PASS : 104160ns
2031165000 : [MON] Stop bit detected (tx=1)
2031165000 : [MON] DUT_TX_data = 0xee(238)
2031165000 : [SCB] PASS !! Expected = 0xee(238), GOT Tx_data = 0xee(238)
    
```

[그림 4-5] UART TX 시나리오2(Random)

(3) Scoreboard 결과

UART TX 검증은 Generator 기반 expected data와 Monitor가 복원한 UART TX output 데이터를 Scoreboard에서 비교하여 수행됩니다.

```
-----  
** UART TX Verification **  
** total test num = 500 **  
** pass test num  = 500 **  
** fail test num  =  0 **  
*****
```

[그림 4-6] UART TX Scoreboard 결과 (PASS)

Scoreboard 기준 모든 transaction이 일치하여 UART TX 기능이 정상적으로 동작함을 확인하였습니다.

4.3 FIFO Simulation

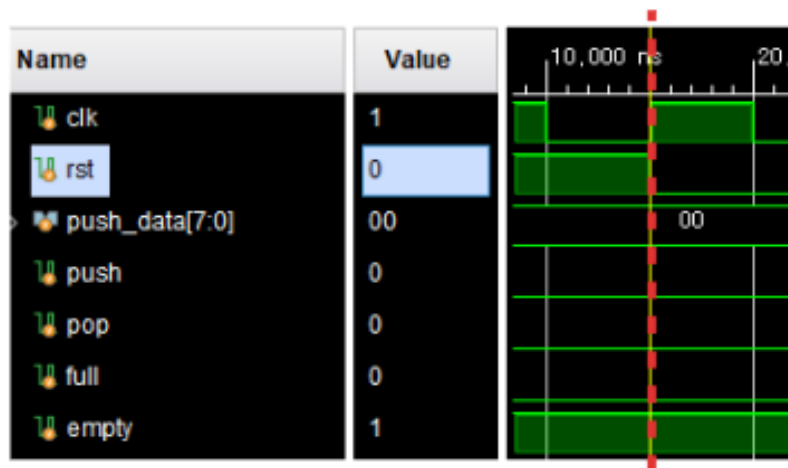
FIFO 검증은 Preset, Only push, Only pop, Randomize의 4가지 시나리오로 구성됩니다.

항목	시나리오
Preset	reset 이후 empty/full 상태 및 초기 동작 확인
Only Push	push=1, pop=0 조건에서 FIFO write 및 full 상태 검증
Only Pop	push=0, pop=1 조건에서 FIFO read 및 empty 상태 검증
Randomize	push/pop 혼합 조건에서 FIFO 동작 및 데이터 정합성 검증

(1) Preset 시나리오 결과

Reset 이후 FIFO 초기 상태를 확인하며 empty/full 플래그 및 내부 데이터 상태가 정상적으로 초기화되는지 검증합니다.

FIFO는 초기 상태에서 empty=1, full=0을 유지하며 데이터가 존재하지 않는 상태로 동작합니다.



```
[DRV Assert] reset pass : empty!
[DRV Assert] reset pass : full!
```

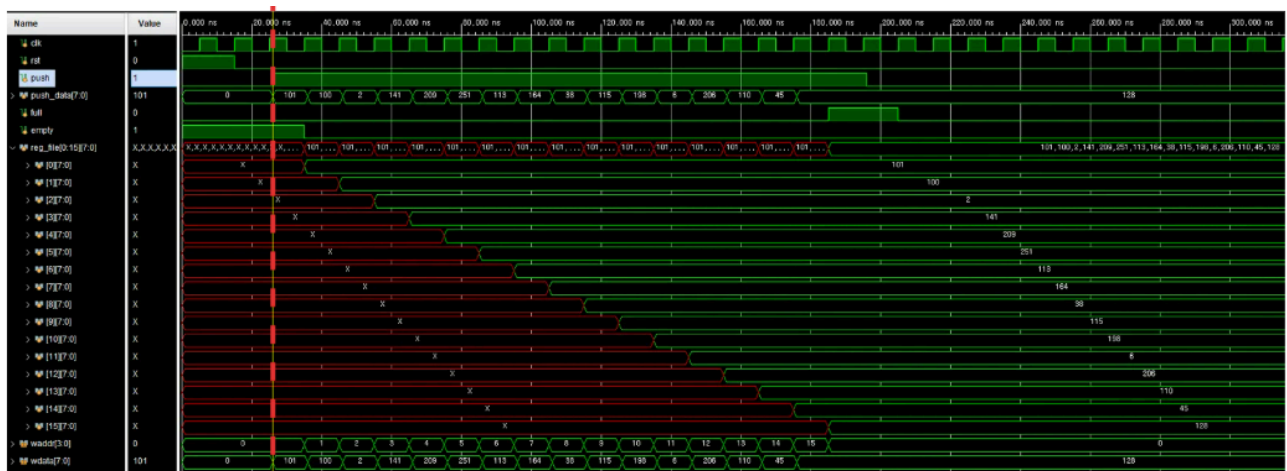
[그림 4-7] FIFO 시나리오1(Preset)

(2) Only Push 시나리오

push=1, pop=0 조건에서 연속 push 동작을 수행하여 FIFO write 기능을 검증합니다.

클럭 상승 엣지 기준으로 입력 데이터가 FIFO 내부 메모리에 순차적으로 저장되며 write pointer가 증가하는 것을 확인합니다.

최종적으로 FIFO full 상태가 정상적으로 assert되어 write 경로의 정상 동작을 검증합니다.



fifo push only test

```

26000 : [DRV] push = 1, pop = 0, push_data = 101, full = 0, empty = 1
36000 : [DRV] push = 1, pop = 0, push_data = 100, full = 0, empty = 0
46000 : [DRV] push = 1, pop = 0, push_data = 2, full = 0, empty = 0
56000 : [DRV] push = 1, pop = 0, push_data = 141, full = 0, empty = 0
66000 : [DRV] push = 1, pop = 0, push_data = 209, full = 0, empty = 0
76000 : [DRV] push = 1, pop = 0, push_data = 251, full = 0, empty = 0
86000 : [DRV] push = 1, pop = 0, push_data = 113, full = 0, empty = 0
96000 : [DRV] push = 1, pop = 0, push_data = 164, full = 0, empty = 0
106000 : [DRV] push = 1, pop = 0, push_data = 38, full = 0, empty = 0
116000 : [DRV] push = 1, pop = 0, push_data = 115, full = 0, empty = 0
126000 : [DRV] push = 1, pop = 0, push_data = 198, full = 0, empty = 0
136000 : [DRV] push = 1, pop = 0, push_data = 6, full = 0, empty = 0
146000 : [DRV] push = 1, pop = 0, push_data = 206, full = 0, empty = 0
156000 : [DRV] push = 1, pop = 0, push_data = 110, full = 0, empty = 0
166000 : [DRV] push = 1, pop = 0, push_data = 45, full = 0, empty = 0
176000 : [DRV] push = 1, pop = 0, push_data = 128, full = 0, empty = 0
176000 : [ENV] PUSH_ONLY TASK END
    
```

[DRV Assert] FIFO FULL, PASS : push only 16 time test

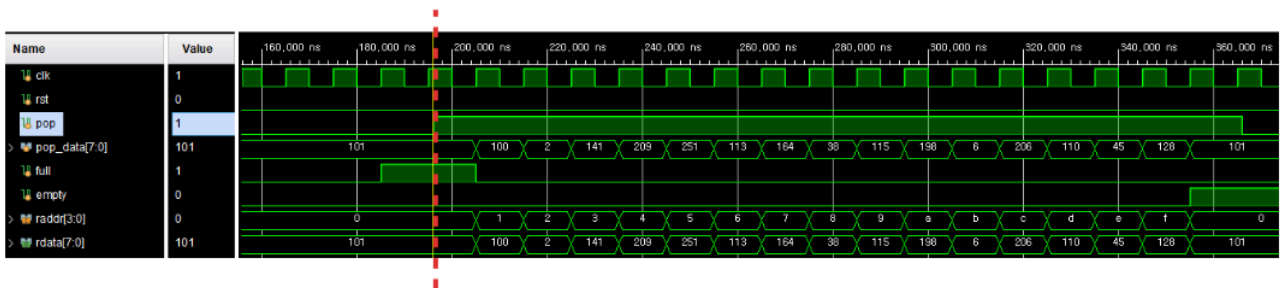
[그림 4-8] FIFO 시나리오2(Push only)

(3) Only Pop 시나리오

push=0, pop=1 조건에서 연속 pop 동작을 수행하여 FIFO read 기능을 검증합니다.

FIFO 내부에 저장된 데이터가 순차적으로 pop_data로 출력되며 read pointer가 증가하는 것을 확인합니다.

모든 데이터 출력 이후 FIFO empty 상태가 정상적으로 assert되어 read 경로의 정상 동작을 검증합니다.



[그림 4-9] FIFO 시나리오3(Pop only)

(4) Randomize 시나리오

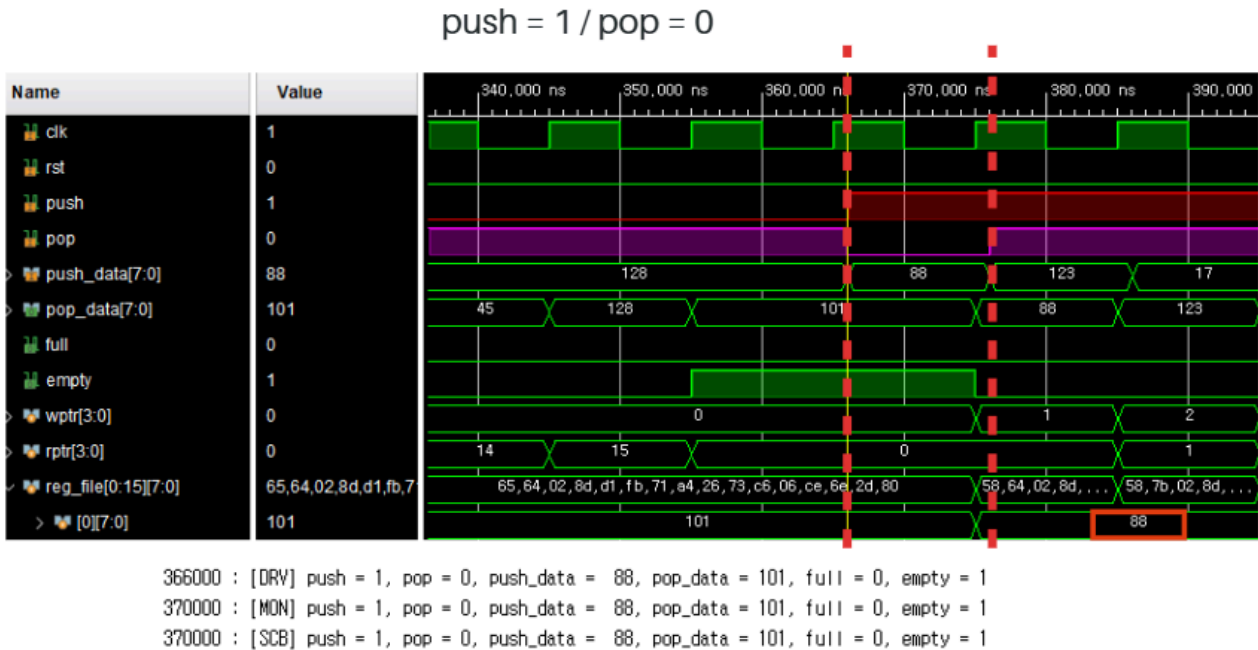
Randomize 시나리오는 push/pop 조합을 4가지 상태로 분리하여 FIFO의 모든 동작 상태를 검증합니다.

항목	시나리오
S0	push=1, pop=0 (Write Only)
S1	push=0, pop=1 (Read Only)
S2	push=1, pop=1 (Concurrent Read/Write)
S3	push=0, pop=0 (Idle 상태 유지)

- S0 (Write Only)

posedge clk + 1ns 이후 push 신호를 인가하여 race condition을 방지하도록 구성하였습니다.

해당 타이밍에서 push=1, pop=0 조건으로 FIFO write 동작을 수행하며, push_data(88)가 FIFO 내부 reg_file에 정상적으로 저장되는 것을 확인하였습니다.



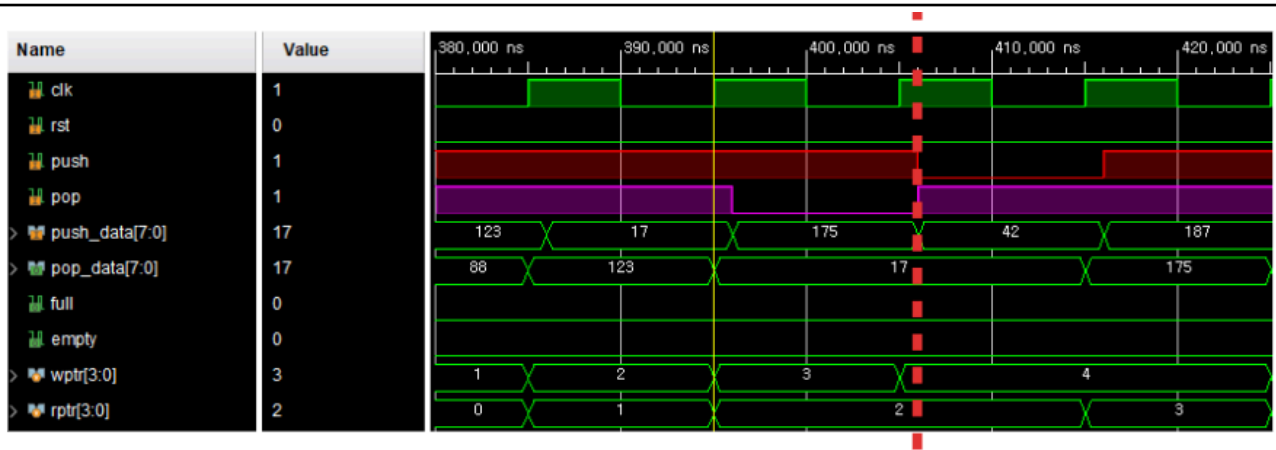
[그림 4-10] FIFO 시나리오4(Push = 1/Pop = 0)

push = 1, pop = 0, write data = 88, reg_file 데이터 저장 확인, write pointer 정상 증가 확인

- S1 (Read Only)

posedge clk + 1ns 이후 pop 신호를 인가하여 race condition을 방지하도록 구성하였습니다.

해당 타이밍에서 push=0, pop=1 조건으로 FIFO read 동작을 수행하며, FIFO 내부에 저장된 데이터가 pop_data(17)로 정상 출력되는 것을 확인하였습니다.



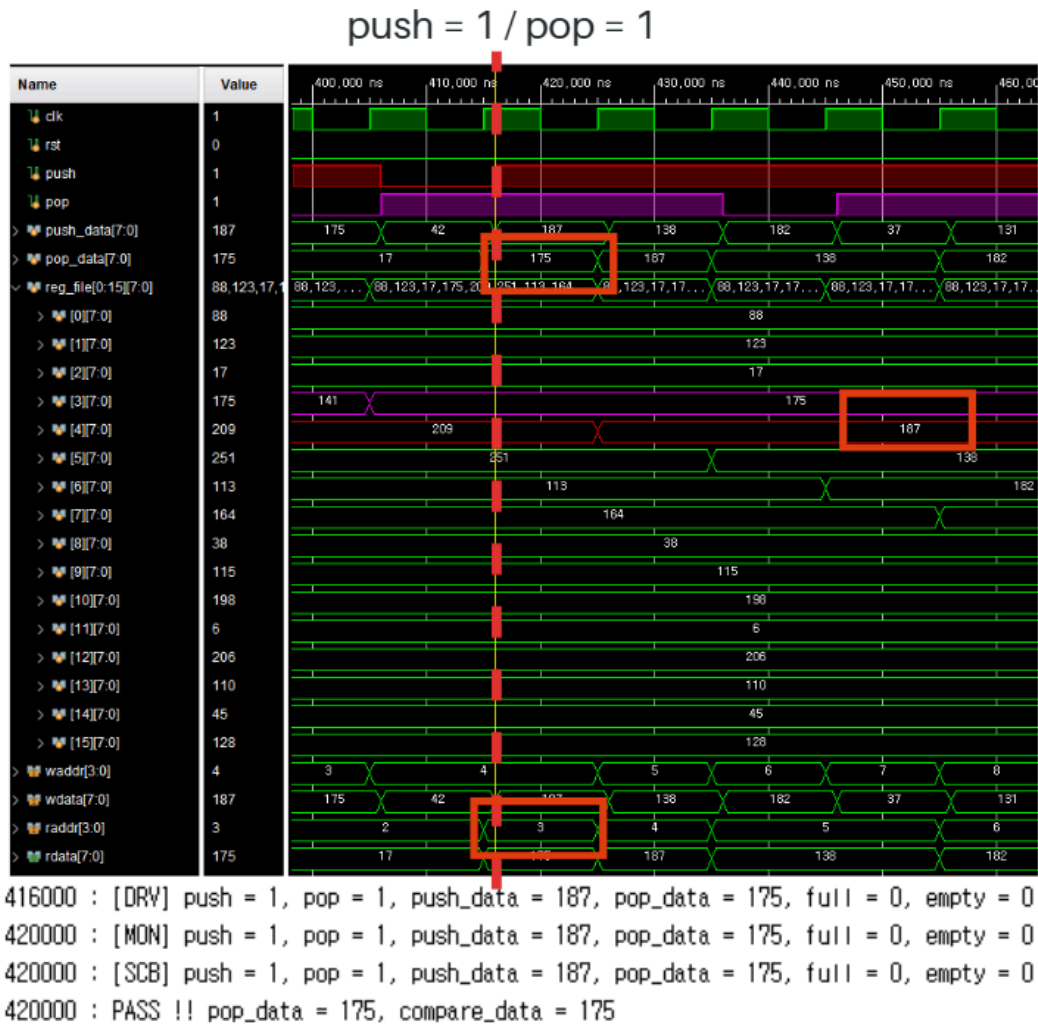
406000 : [DRV] push = 0, pop = 1, push_data = 42, pop_data = 17, full = 0, empty = 0
 410000 : [MON] push = 0, pop = 1, push_data = 42, pop_data = 17, full = 0, empty = 0
 410000 : [SCB] push = 0, pop = 1, push_data = 42, pop_data = 17, full = 0, empty = 0

[그림 4-11] FIFO 시나리오4(Push = 0/Pop = 1)

push = 0, pop = 1, read data = 17 (pop_data), pop_data 정상 출력 확인, read pointer 정상 증가 확인

- S2 (Read/Write)

posedge clk + 1ns 이후 push 신호와 pop 신호를 동시에 인가하여 race condition을 방지하도록 구성하였습니다. 해당 타이밍에서 push=1, pop=1 조건으로 FIFO의 read 및 write 동작을 동시에 수행하며, push_data(187)이 FIFO 내부 reg_file에 정상적으로 저장되고 pop_data(175)가 정상 출력되는 것을 확인하였습니다.



[그림 4-12] FIFO 시나리오4(Push = 1/Pop = 1)

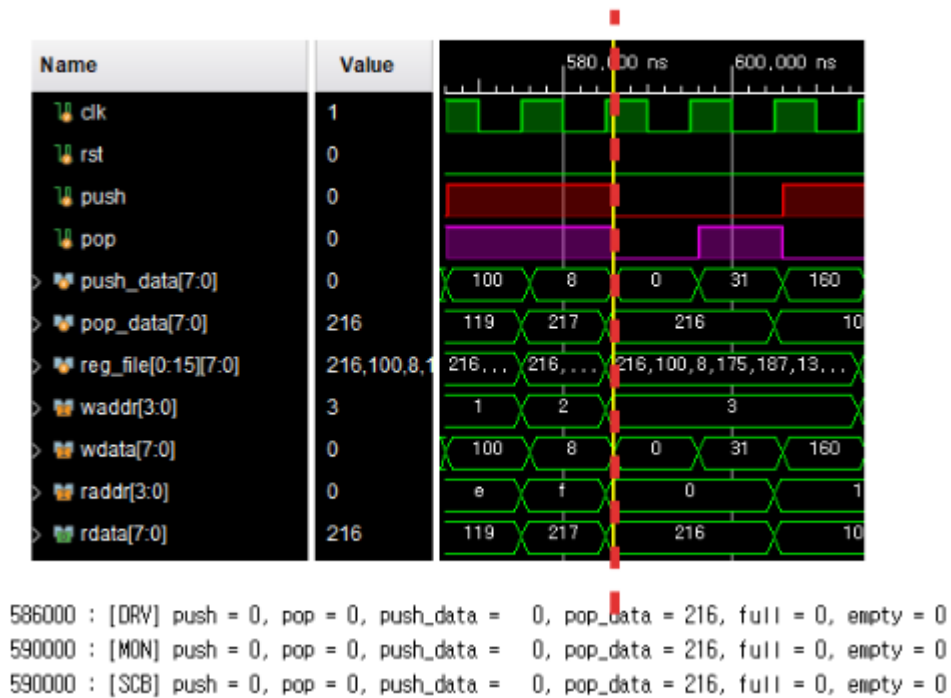
push = 1, pop = 1, write data = 187, read data = 175 (pop_data)

,reg_file 데이터 저장 확인, pop_data 정상 출력 확인

- S3 (Idle)

posedge clk + 1ns 이후 push와 pop 신호를 모두 비활성화하여 FIFO의 대기 상태를 확인하였습니다. 데이터 입력 및 출력이 발생하지 않는 상태에서 FIFO 상태 정보가 정상적으로 유지되는 것을 확인하였습니다.

push = 0 / pop = 0



[그림 4-13] FIFO 시나리오4(Push = 0/Pop = 0)

push = 0, pop = 0, 데이터 입력 없음, 데이터 출력 없음, FIFO 상태 유지 확인, full/empty 상태 정상 유지 확인

(5) Scoreboard 결과

Randomize 시나리오에서는 Scoreboard를 이용하여 FIFO 출력 데이터의 정확성을 검증하였습니다. Scoreboard는 push 동작 시 입력 데이터를 Queue에 저장하고, pop 동작 시 Queue에 저장된 기대 데이터와 실제 pop_data를 비교하여 FIFO의 선입선출(FIFO) 동작을 검증합니다.

검증 결과 총 500개의 Transaction을 수행하였으며, pop 동작이 발생한 344건에 대해 데이터 비교가 수행되었습니다. 모든 비교 결과가 일치하여 FIFO가 데이터 손실 없이 정상 동작함을 확인하였습니다.

```

-----
** FIFO IP Verification **
** total test num = 500 **
** pass test num  = 344 **
** fail test num  =  0 **
*****
    
```

[그림 4-14] FIFO Scoreboard 결과 (PASS)

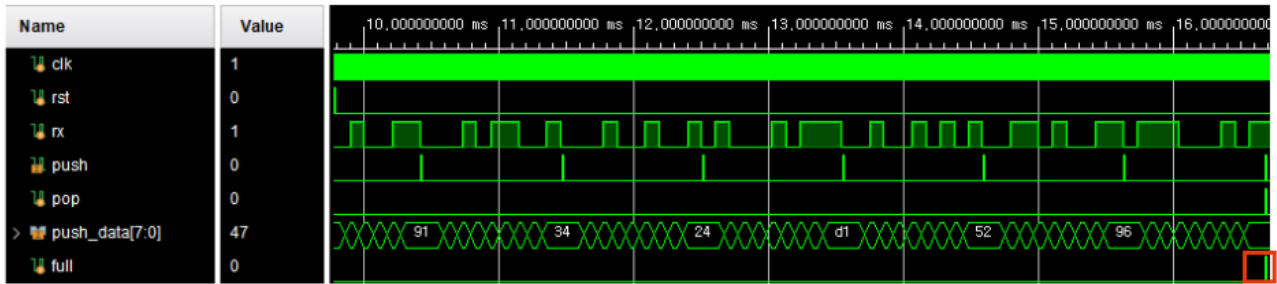
4.4 UART RX + FIFO RX Simulation

UART RX + FIFO RX 검증은 Preset 시나리오와 Randomize 시나리오로 구성하여 UART 수신 데이터의 FIFO 저장 기능과 데이터 정합성을 검증합니다.

항목	시나리오
FIFO FULL	랜덤 데이터 16개 수신 후 FIFO FULL 상태 확인
POP & 검증	FIFO 데이터 출력 및 데이터 정합성 검증

(1) FIFO FULL 시나리오 결과

UART 프로토콜 형식으로 랜덤 데이터 16개를 rx 신호에 순차적으로 인가하였습니다. UART RX는 수신 데이터를 병렬 데이터로 복원하여 FIFO에 저장하며, FIFO가 가득 찬 이후 full 신호가 정상적으로 assert 되는 것을 확인하였습니다.



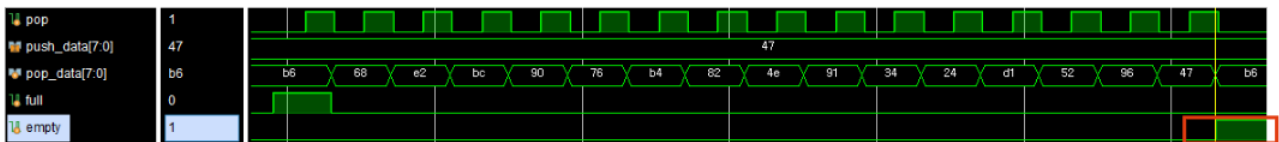
PASS : FIFO FULL

[그림 4-15] UART RX + FIFO RX 시나리오1(FIFO FULL)

(2) POP & 검증 시나리오 결과

FIFO에 저장된 데이터를 출력하기 위해 Driver에서 pop 신호를 16회 인가하였습니다. Monitor는 출력된 pop_data를 수집하여 Scoreboard로 전달하고, Scoreboard는 UART RX 수신 데이터와 FIFO 출력 데이터를 비교하여 데이터 정합성을 검증하였습니다.

검증 결과 FIFO에 저장된 데이터가 입력 순서와 동일하게 출력되었으며, 데이터 손실 없이 FIFO의 선입선출(FIFO) 동작이 정상적으로 수행되는 것을 확인하였습니다.



PASS : FIFO EMPTY

[그림 4-16] UART RX + FIFO RX 시나리오2(POP & 검증)

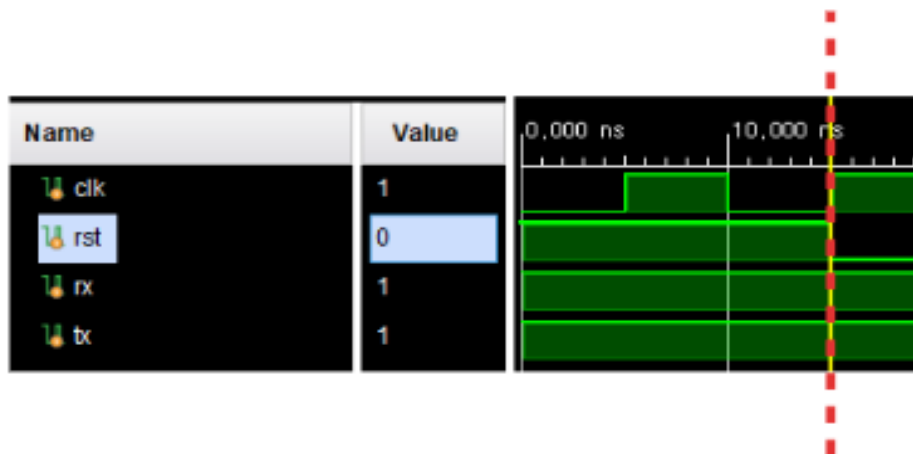
4.5 UART + FIFO Loopback Simulation

UART + FIFO Loopback 검증은 UART RX로 입력된 데이터가 FIFO를 거쳐 UART TX로 정상 출력되는지 확인하기 위해 수행하였습니다.

항목	시나리오
Preset	Reset 이후 UART RX 및 UART TX Idle 상태 확인
Randomize	랜덤 데이터 500개 전송 후 데이터 정합성 검증
Sequence	1~16 순차 데이터 전송 후 출력 순서 검증
Boundary	경계값 데이터 전송 후 데이터 정합성 검증

(1) Preset 시나리오 결과

Reset 이후 UART RX와 UART TX의 초기 상태를 확인하였습니다. UART RX 입력 신호와 UART TX 출력 신호 모두 Idle 상태를 유지하며 정상 초기화되는 것을 확인하였습니다.



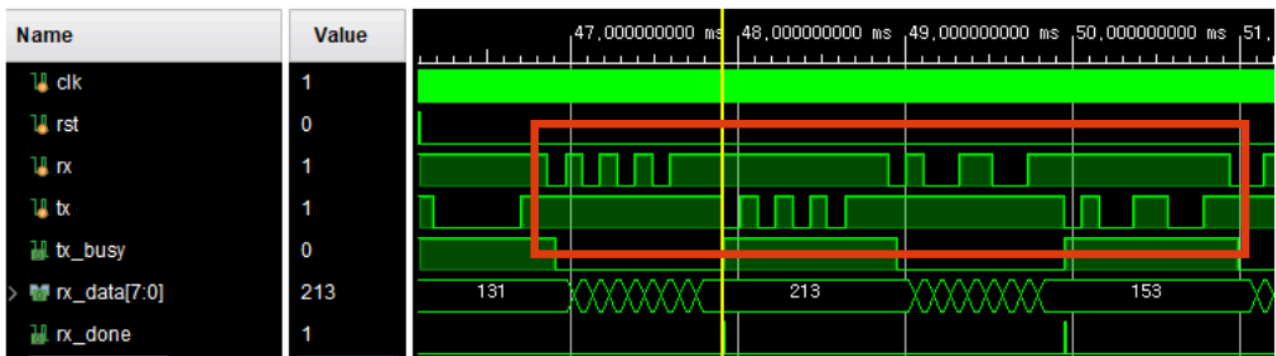
```
[DRV Assert] reset pass : rx = 1 (idle)
[DRV Assert] reset pass : tx = 1 (idle)
```

[그림 4-17] UART + FIFO LOOPBACK 시나리오1(Preset)

(2) Randomize 시나리오 결과

Generator에서 생성한 랜덤 데이터 500개를 UART RX 입력으로 전송하였습니다. 입력된 데이터는 UART RX에서 병렬 데이터로 복원된 후 FIFO에 저장되며, FIFO TX를 통해 다시 UART TX로 전달됩니다. Monitor는 UART TX 출력 신호를 복원하여 Scoreboard로 전달하였습니다.

Scoreboard는 Generator에서 생성한 원본 데이터와 UART TX에서 최종 출력된 데이터를 비교하여 전체 Loopback 경로의 데이터 정합성을 검증하였습니다. 검증 결과 모든 데이터가 입력값과 동일하게 출력되었으며, UART RX → FIFO RX → FIFO TX → UART TX 경로에서 데이터 손실, 순서 오류 및 변형 없이 정상적으로 전달되는 것을 확인하였습니다.



[그림 4-18] UART + FIFO LOOPBACK 시나리오2(Randomize)

```

-----
** UART FIFO Verification **
** total test num = 500 **
** pass test num = 500 **
** fail test num = 0 **
*****
    
```

[그림 4-19] UART + FIFO LOOPBACK 시나리오2 Scoreboard 결과 (PASS)

(3) Sequence 시나리오 결과

1부터 16까지의 순차 데이터를 UART RX 입력으로 전송하였습니다. 입력된 데이터는 UART RX → FIFO RX → FIFO TX → UART TX 경로를 통해 전달되며, UART TX 출력 데이터를 확인한 결과 입력 순서와 동일한 순서로 출력되는 것을 확인하였습니다.

이를 통해 FIFO의 선입선출(FIFO) 특성이 정상적으로 유지되며, 전체 Loopback 경로에서 데이터 순서가 보존됨을 검증하였습니다.

```
sequence 1~16 test
101881575000 : [GEN] RX_data = 0x01(1)
101881575000 : [DRV] RX_data = 0x01(1)
103919185000 : [MON] TX_data = 0x01(1)
103919185000 : [SCB] RX_data = 0x01(1), TX_data = 0x00(0)
103919185000 : [SCB] RX_data = 0x00(0), TX_data = 0x01(1)
103919185000 : [SCB] PASS !! expected_data = 1, got tx_data = 1
103919185000 : [GEN] RX_data = 0x02(2)
103919185000 : [DRV] RX_data = 0x02(2)
105956815000 : [MON] TX_data = 0x02(2)
105956815000 : [SCB] RX_data = 0x02(2), TX_data = 0x00(0)
105956815000 : [SCB] RX_data = 0x00(0), TX_data = 0x02(2)
105956815000 : [SCB] PASS !! expected_data = 2, got tx_data = 2
105956815000 : [GEN] RX_data = 0x03(3)
105956815000 : [DRV] RX_data = 0x03(3)
107994445000 : [MON] TX_data = 0x03(3)
107994445000 : [SCB] RX_data = 0x03(3), TX_data = 0x00(0)
107994445000 : [SCB] RX_data = 0x00(0), TX_data = 0x03(3)
107994445000 : [SCB] PASS !! expected_data = 3, got tx_data = 3
107994445000 : [GEN] RX_data = 0x04(4)
107994445000 : [DRV] RX_data = 0x04(4)
110032075000 : [MON] TX_data = 0x04(4)
110032075000 : [SCB] RX_data = 0x04(4), TX_data = 0x00(0)
110032075000 : [SCB] RX_data = 0x00(0), TX_data = 0x04(4)
110032075000 : [SCB] PASS !! expected_data = 4, got tx_data = 4
110032075000 : [GEN] RX_data = 0x05(5)
122257855000 : [GEN] RX_data = 0x0b(11)
122257855000 : [DRV] RX_data = 0x0b(11)
124295485000 : [MON] TX_data = 0x0b(11)
124295485000 : [SCB] RX_data = 0x0b(11), TX_data = 0x00(0)
124295485000 : [SCB] RX_data = 0x00(0), TX_data = 0x0b(11)
124295485000 : [SCB] PASS !! expected_data = 11, got tx_data = 11
124295485000 : [GEN] RX_data = 0x0c(12)
124295485000 : [DRV] RX_data = 0x0c(12)
126333115000 : [MON] TX_data = 0x0c(12)
126333115000 : [SCB] RX_data = 0x0c(12), TX_data = 0x00(0)
126333115000 : [SCB] RX_data = 0x00(0), TX_data = 0x0c(12)
126333115000 : [SCB] PASS !! expected_data = 12, got tx_data = 12
126333115000 : [GEN] RX_data = 0x0d(13)
126333115000 : [DRV] RX_data = 0x0d(13)
128370745000 : [MON] TX_data = 0x0d(13)
128370745000 : [SCB] RX_data = 0x0d(13), TX_data = 0x00(0)
128370745000 : [SCB] RX_data = 0x00(0), TX_data = 0x0d(13)
128370745000 : [SCB] PASS !! expected_data = 13, got tx_data = 13
128370745000 : [GEN] RX_data = 0x0e(14)
128370745000 : [DRV] RX_data = 0x0e(14)
130408375000 : [MON] TX_data = 0x0e(14)
130408375000 : [SCB] RX_data = 0x0e(14), TX_data = 0x00(0)
130408375000 : [SCB] RX_data = 0x00(0), TX_data = 0x0e(14)
130408375000 : [SCB] PASS !! expected_data = 14, got tx_data = 14
130408375000 : [GEN] RX_data = 0x0f(15)
130408375000 : [DRV] RX_data = 0x0f(15)
132446005000 : [MON] TX_data = 0x0f(15)
132446005000 : [SCB] RX_data = 0x0f(15), TX_data = 0x00(0)
132446005000 : [SCB] RX_data = 0x00(0), TX_data = 0x0f(15)
132446005000 : [SCB] PASS !! expected_data = 15, got tx_data = 15
132446005000 : [GEN] RX_data = 0x10(16)
132446005000 : [DRV] RX_data = 0x10(16)
134483635000 : [MON] TX_data = 0x10(16)
134483635000 : [SCB] RX_data = 0x10(16), TX_data = 0x00(0)
134483635000 : [SCB] RX_data = 0x00(0), TX_data = 0x10(16)
134483635000 : [SCB] PASS !! expected_data = 16, got tx_data = 16
134483635000 : [ENV] SEQUENCE TASK END
```

[그림 4-20] UART + FIFO LOOPBACK 시나리오3(Sequence)

(4) Boundary 시나리오 결과

UART 통신에서 대표적인 경계값 데이터인 0xFF, 0x55, 0xAA, 0x00을 UART RX 입력으로 전송하였습니다. 입력된 데이터는 Loopback 경로를 통해 UART TX로 출력되며, 출력 데이터를 확인한 결과 모든 데이터가 정상적으로 일치하는 것을 확인하였습니다.

특히 모든 비트가 1인 데이터(0xFF), 모든 비트가 0인 데이터(0x00), 비트 토글 패턴 데이터(0x55, 0xAA)에 대해서도 데이터 손실이나 변형 없이 정상적으로 전송됨을 검증하였습니다.

```

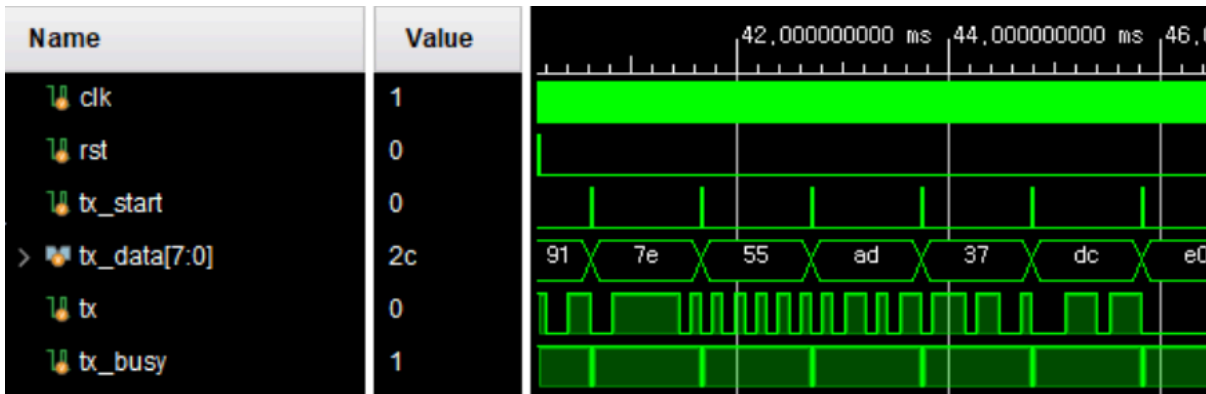
Boundary (FF, 55, AA, 00)
134483655000 : [GEN] RX_data = 0xff(255)
134483655000 : [DRV] RX_data = 0xff(255)
136521265000 : [MON] TX_data = 0xff(255)
136521265000 : [SCB] RX_data = 0xff(255), TX_data = 0x00(0)
136521265000 : [SCB] RX_data = 0x00(0), TX_data = 0xff(255)
136521265000 : [SCB] PASS !! expected_data = 255, got tx_data = 255
136521265000 : [GEN] RX_data = 0x55(85)
136521265000 : [DRV] RX_data = 0x55(85)
138558895000 : [MON] TX_data = 0x55(85)
138558895000 : [SCB] RX_data = 0x55(85), TX_data = 0x00(0)
138558895000 : [SCB] RX_data = 0x00(0), TX_data = 0x55(85)
138558895000 : [SCB] PASS !! expected_data = 85, got tx_data = 85
138558895000 : [GEN] RX_data = 0xaa(170)
138558895000 : [DRV] RX_data = 0xaa(170)
140596525000 : [MON] TX_data = 0xaa(170)
140596525000 : [SCB] RX_data = 0xaa(170), TX_data = 0x00(0)
140596525000 : [SCB] RX_data = 0x00(0), TX_data = 0xaa(170)
140596525000 : [SCB] PASS !! expected_data = 170, got tx_data = 170
140596525000 : [GEN] RX_data = 0x00(0)
140596525000 : [DRV] RX_data = 0x00(0)
142634155000 : [MON] TX_data = 0x00(0)
142634155000 : [SCB] RX_data = 0x00(0), TX_data = 0x00(0)
142634155000 : [SCB] RX_data = 0x00(0), TX_data = 0x00(0)
142634155000 : [SCB] PASS !! expected_data = 0, got tx_data = 0
142634155000 : [ENV] BOUNDARY TASK END
142634155000 : [ENV] ALL TASK END

```

[그림 4-21] UART + FIFO LOOPBACK 시나리오 4(Boundary)

4.6 트러블슈팅 (Trouble Shooting)

UART TX 검증 중 첫 번째 **drive**는 정상 동작하였으나 두 번째 **drive** 이후 시뮬레이션 전체가 정지하는 현상이 발생하였습니다.



[그림 4-22] tx_busy 동기화 누락에 따른 시뮬레이션 정지(before) 및 wait(!tx_busy) 적용 결과(after)

항목	내용
문제 현상	첫 번째 데이터 전송은 정상적으로 수행되었으나, 두 번째 drive 이후 Monitor 가 응답하지 않으며 시뮬레이션이 정지하는 현상이 발생하였습니다.
원인 분석	UART TX 가 데이터 전송 중(tx_busy=1)인 상태에서 다음 tx_start 신호를 인가하였습니다. DUT 는 해당 요청을 무시하였고 UART TX 출력 파형이 발생하지 않아 Monitor 가 UART 프레임을 무한 대기하게 되었습니다.
해결 방안	Driver 에 wait(!uart_vif.tx_busy) 구문을 추가하여 이전 UART 전송 완료를 확인한 후 다음 데이터를 전송하도록 수정하였습니다.
결과	모든 Transaction 이 정상적으로 전송되었으며 Monitor 와 Scoreboard 가 정상 동작하여 시뮬레이션 정지 문제가 해결되었습니다.

6. 검증 결과 요약

검증 시나리오	검증 내용	테스트 수	PASS	FAIL	달성 여부
UART RX	RX_Data Random 전송 → rx_done 및 rx_data 정합성 확인	500	500	0	달성 ✓
UART TX	TX_Data Random 전송 → 직렬 전송 복원 데이터 정합성 확인	500	500	0	달성 ✓
FIFO (Random)	Push/Pop Random 수행 → FIFO 데이터 순서 보존 확인	500	344	0	달성 ✓
UART RX + FIFO RX	RX_Data → Pop 시 Pop_Data == RX_Data 확인	16	16	0	달성 ✓
UART + FIFO LOOPBACK	RX_Data → TX_Data 출력 일치 확인	500	500	0	달성 ✓

- FIFO(Random) 의 PASS=344는 실제 pop이 발생하여 Scoreboard 비교가 수행된 횟수입니다.
- UART RX + FIFO RX 는 FIFO Full(16개 저장) 후 Pop 16회 수행하여 검증한 결과입니다.

7. 결론 및 고찰

본 프로젝트에서는 UART RX, UART TX 및 FIFO 모듈에 대한 SystemVerilog 기반 검증 환경을 구축하였으며, 단위 모듈 검증과 UART RX → FIFO → UART TX로 구성된 Loopback 통합 검증을 수행하였습니다. 이를 통해 UART 송수신 기능, FIFO 데이터 저장 및 출력 기능, 데이터 정합성 및 순서 보존 특성이 정상적으로 동작함을 확인하였습니다.

- 자동화 검증 환경 구축 : Generator, Driver, Monitor, Scoreboard 구조를 적용하여 랜덤 데이터 생성, DUT 입력, 출력 데이터 수집 및 기대값 비교가 자동으로 수행되는 검증 환경을 구현하였습니다.
- UART RX/TX 검증 완료 : UART RX와 UART TX에 대해 각각 500개의 랜덤 데이터를 이용한 검증을 수행하였으며, 모든 데이터가 정상적으로 수신 및 송신되는 것을 확인하였습니다.
- FIFO 기능 검증 완료 : FIFO Full 검증과 Random Push/Pop 검증을 수행하여 데이터 저장 및 출력 기능, Full 상태 검출 기능, 선입선출(FIFO) 특성이 정상적으로 동작함을 확인하였습니다.
- 통합 검증 수행 : UART RX + FIFO RX 검증과 UART + FIFO Loopback 검증을 수행하여 UART RX → FIFO → UART TX 전체 경로에서 데이터 손실 및 순서 오류 없이 데이터가 정상적으로 전달됨을 확인하였습니다.
- 검증 환경 신뢰성 확보 : Driver 구동 시점을 `posedge clk + 1ns`로 설계하여 `race condition`을 방지하였으며, UART TX 검증 과정에서 발생한 `tx_busy` 동기화 문제를 해결함으로써 DUT와 검증 환경 간의 안정적인 동작을 확보하였습니다.

본 프로젝트를 통해 단위 모듈 검증뿐만 아니라 시스템 수준의 통합 검증 절차를 경험할 수 있었으며, 자동화된 검증 환경이 검증 효율성과 신뢰성 향상에 효과적임을 확인할 수 있었습니다. 또한 다양한 시나리오 기반 검증을 통해 UART 및 FIFO 시스템의 기능적 신뢰성을 확보할 수 있었습니다.

참고 문헌

- [0] IEEE Standard for SystemVerilog - Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2017
- [1] UART Protocol Specification, Texas Instruments Application Report
- [2] Basys 3 FPGA Board Reference Manual, Digilent Inc.